

Desafio Tech: Projeto "OmniConnect – Gestão Inteligente de Serviços"

O Cenário

Muitas empresas de serviços (clínicas, escritórios de advocacia, oficinas premium) sofrem com dois problemas:

1. **Fricção no Atendimento:** Clientes preferem o WhatsApp, mas os dados se perdem em conversas informais.
2. **Falta de Visibilidade:** O prestador de serviço não consegue gerir a demanda e o cliente não tem um histórico estruturado do que foi tratado.

O Objetivo: Criar uma plataforma onde o cliente inicia o atendimento via **WhatsApp (Chatbot com IA)**, os dados são processados e sincronizados em tempo real, e ambos (Cliente e Fornecedor) utilizam um **App em Flutter** para gerir o ciclo de vida do serviço.

Requisitos Técnicos (Obrigatórios)

1. **Frontend (Mobile/Web):** Aplicação em **Flutter** com dois perfis de acesso:
 - **Cliente:** Visualiza status, histórico de chats, documentos e recebe notificações.
 - **Fornecedor:** Dashboard de gestão, controle de agenda e interação com os dados extraídos pela IA.
 2. **Backend:** API robusta em **Node.js** ou **FastAPI (Python)**.
 3. **Inteligência Artificial:**
 - Uso de **LangChain** para orquestração.
 - Implementação de **RAG (Retrieval-Augmented Generation)** para que o bot responda com base em documentos técnicos ou regras de negócio da empresa.
 - Integração com **WhatsApp Business API (Cloud API)**.
 4. **Infraestrutura e Dados:**
 - Banco de dados: **MongoDB** ou **PostgreSQL**.
 - Notificações Push: **Firebase Cloud Messaging (FCM)**.
 - Hospedagem: Recomendado o uso de containers (Docker/LXC).
-

Cronograma de Execução (4 Semanas)

Semana 1: Fundação e Conectividade

- **Back-end:** Setup da API e conexão com o banco de dados.
- **WhatsApp:** Configuração do Webhook da Cloud API e "Hello World" de mensagens.
- **Flutter:** Estrutura base do App (Navegação, Auth e Temas).
- **Entrega:** Webhook recebendo e logando mensagens do WhatsApp no banco.

Semana 2: O "Cérebro" (IA e RAG)

- **LangChain:** Implementação da lógica de atendimento.
- **RAG:** Criação do Vector Database (ex: Pinecone, Chroma ou PGVector) com os documentos de conhecimento da empresa.
- **Sincronização:** Toda interação no WhatsApp deve atualizar o status no banco de dados para o App Flutter.
- **Entrega:** Bot respondendo dúvidas complexas via WhatsApp com base em documentos.

Semana 3: Experiência do Usuário (Flutter + Firebase)

- **App Cliente:** Tela de acompanhamento e histórico.
- **App Fornecedor:** Painel de controle de atendimentos.
- **Firebase:** Configuração do FCM para que, quando o bot finalizar uma triagem no WhatsApp, o fornecedor receba uma notificação "Novo Serviço Pendente".
- **Entrega:** Apps funcionalmente integrados ao banco de dados.

Semana 4: Refino, Testes e Pitch

- Tratamento de erros e casos de borda (mensagens de áudio, imagens).
- Documentação da API (Swagger/Postman).
- Preparação de uma demo de 10 minutos simulando o fluxo ponta a ponta.
- **Entrega Final:** Código no GitHub, Documentação e Apresentação.

Formação dos Grupos (5 Integrantes)

Sugestão de papéis para os alunos:

1. **Tech Lead / Backend:** Responsável pela arquitetura da API e segurança.
2. **AI Specialist:** Focado em LangChain, RAG e integração com LLM.
3. **Flutter Developer (Client-side):** UI/UX e consumo da API para o cliente.
4. **Flutter Developer (Provider-side):** Dashboards e regras de negócio do fornecedor.
5. **DevOps & QA:** Firebase, notificações, deploy (LXC/Docker) e testes integrados.

🎯 Critérios de Avaliação

- **Integração Real:** O dado enviado no WhatsApp aparece no App Flutter em < 2 segundos?
- **Qualidade da IA:** O uso das tools (MCP, RAG, SKILLS, PIPELINE) está evitando alucinações e retornando informações relevantes da base de conhecimento?
- **UX/UI:** O App é intuitivo para um usuário leigo?
- **Robustez:** O sistema trata mensagens que não são texto (ex: o usuário manda uma foto)?

Dica: "Não foquem apenas no código, mas em como a IA reduz o trabalho manual do fornecedor ao extrair dados importantes da conversa de forma automática."